

Project Title

Combating Crime Through Time Series Analysis

School of Computer Science Engineering and Technology



Bennett University
Greater Noida, Uttar Pradesh

Submitted by
Shivansh Mishra, E22CSEU1004
Atulya Tripathi, E22CSEU1006

Submitted to
Dr. Gitika Sharma

I. Introduction and Objective

The main objective of this project is to analyse crime trends over time and build predictive models that can help forecast future crime occurrences.

By using time series analysis techniques, we aim to identify patterns and seasonality in crime data, which could help law enforcement agencies allocate resources more efficiently.

The key research question we address is:

"Can historical crime data be used to accurately forecast future crime trends and help predict potential surges or declines?"

II. Data Collection and Preprocessing

Dataset Description:

The dataset used for this project was obtained from the Chicago Police Department's public crime records. It contains detailed information on crimes reported over several years.

Important variables included are:

```
print("\n ♦ Top 10 Crime Affected Districts:")
print(df["District"].value_counts().head(10))

# ♦ Save a small sample (first 100K rows) for further testing
df_sample = df.head(100000)
df_sample.to_csv("crime_data_sample_raw.csv", index=False)

print("\n✅ Analysis Complete! Upload 'crime_data_sample_raw.csv' for review.")
```

```
♦ Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7846809 entries, 0 to 7846808
Data columns (total 22 columns):
#   Column                Dtype
---  -
0   ID                     int64
1   Case Number            object
2   Date                  object
3   Block                 object
4   IUCR                  object
5   Primary Type           object
6   Description            object
7   Location Description   object
8   Arrest                bool
9   Domestic              bool
10  Beat                  int64
11  District               float64
12  Ward                   float64
13  Community Area         float64
14  FBI Code               object
15  X Coordinate            float64
16  Y Coordinate            float64
17  Year                   int64
18  Updated On             object
...
2.0    371765
Name: count, dtype: int64
```

- **Date** of occurrence
- **Primary Type** of crime (e.g., robbery, assault)
- **Latitude and Longitude** for location
- **District, Ward**, and other geographic features
- **Arrest** and **Domestic Violence** indicators

For the time series analysis, we focused specifically on **crime counts** over time. We constructed a **daily and 30-day moving average** based crime time series spanning multiple years.

Preprocessing Steps:

- **Handling Missing Values:** Missing values were checked, and rows with missing critical information like Primary Type, Latitude, and Longitude were dropped to maintain dataset integrity.

```

Dropping Non Informative cols

# Drop unnecessary columns
drop_columns = ["ID", "Case Number", "Updated On", "Location", "X Coordinate", "Y Coordinate"]
df.drop(columns=drop_columns, inplace=True)

print(f"✅ Dropped columns: {drop_columns}")

✅ Dropped columns: ['ID', 'Case Number', 'Updated On', 'Location', 'X Coordinate', 'Y Coordinate']

Handling missing Latitude and Longitude

# Remove rows where both Latitude and Longitude are missing
df = df.dropna(subset=["Latitude", "Longitude"])

print(f"✅ Removed rows with missing Latitude/Longitude: {df.shape}")

✅ Removed rows with missing Latitude/Longitude: (7758698, 16)

Imputing Missing District, Ward, and Community Area

df["District"] = df["District"].ffill()
df["Ward"] = df["Ward"].ffill()
df["Community Area"] = df["Community Area"].ffill()

print(f"✅ Imputed missing values for District, Ward, Community Area")

```

- **Feature Engineering:**
 - Extracted Year, Month, Day, Hour, Season, and Day of the Week from the crime date.
 - Created cyclical features (sin and cos transformations) for Hour and Month.
 - Computed moving averages (Crime_Count_7D, Crime_Count_30D) and differences to capture recent crime trends.

```
# One-Hot Encoding for Season
season_encoded = pd.get_dummies(df['Season'], prefix='Season', drop_first=True)
df = pd.concat([df, season_encoded], axis=1)

# Drop the original column
df.drop('Season', axis=1, inplace=True)

print("\n✅ One-Hot Encoding Applied for Season:")
print(df.columns)

✅ One-Hot Encoding Applied for Season:
Index(['Date', 'Year', 'Month', 'Day', 'Hour', 'DayOfWeek', 'Primary Type',
      'District', 'Community Area', 'Ward', 'Latitude', 'Longitude', 'Arrest',
      'Domestic', 'Weekday', 'WeekOfYear', 'Month_sin', 'Month_cos',
      'Hour_sin', 'Hour_cos', 'Crime_Count_7D', 'Crime_Count_30D',
      'Crime_Count_90D', 'Crime_MA_7D', 'Crime_MA_30D', 'Crime_Diff_7D_30D',
      'Crime_Diff_30D_90D', 'Crime_Cluster', 'Season_Spring', 'Season_Summer',
      'Season_Winter'],
      dtype='object')
```

- **Data Resampling:**
 - The dataset was resampled and filtered to around **250,000 rows** for efficient computation without losing important trends.
- **Normalization/Encoding:**
 - Categorical variables such as Primary Type were encoded for machine learning compatibility.

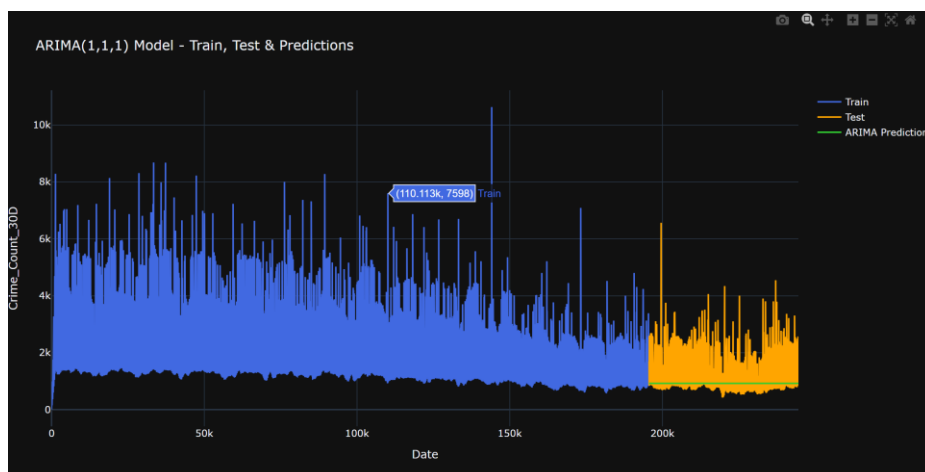
III. Time Series Modeling and Diagnostics

Model Selection and Fitting:

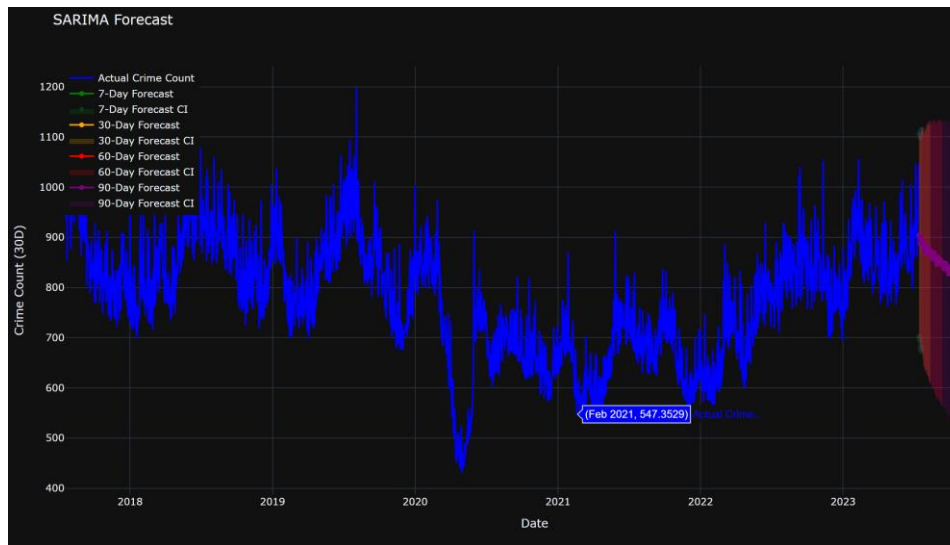
Several models were considered for forecasting crime trends.

Among these, **ARIMA (AutoRegressive Integrated Moving Average)**, **SARIMA (Seasonal ARIMA)**, and **Prophet** were selected.

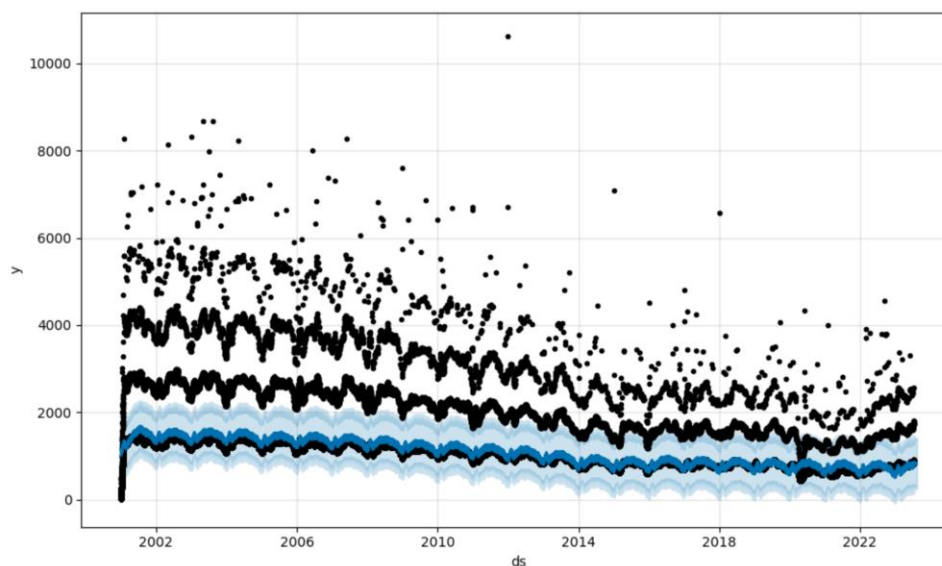
- **ARIMA** was chosen initially because of its strong ability to model non-stationary time series.



- **SARIMA** was selected later to capture both trend and seasonality patterns in crime counts.



- **Prophet**, developed by Facebook, was tried due to its robustness in handling outliers, holidays, and multiple seasonality.



Each model was fitted after analysing the time series properties:

- **Stationarity** was checked using differencing techniques.
- **ACF and PACF plots** were analysed to select appropriate model parameters (p , d , q).

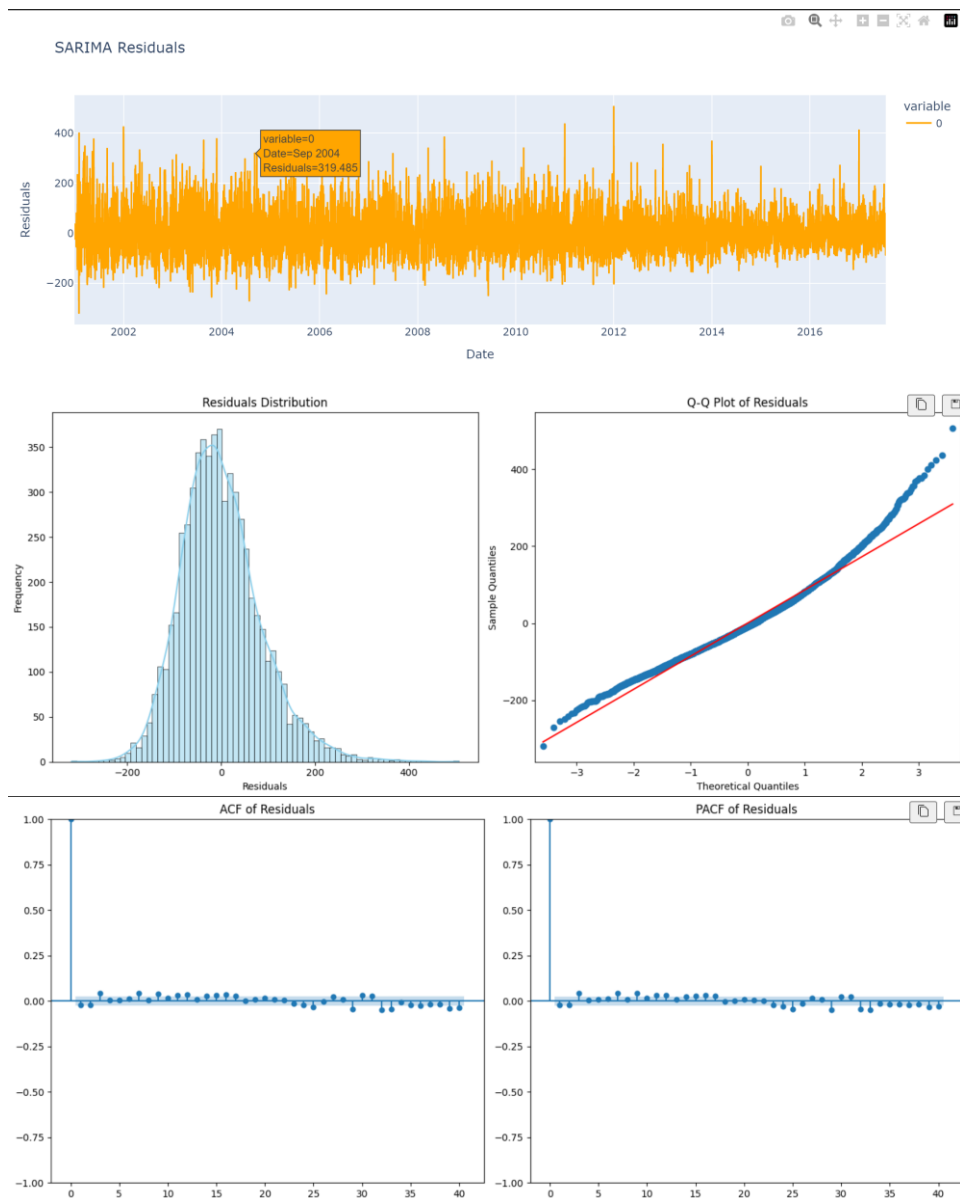
Main Parameters Example (for SARIMA):

- SARIMA (1,1,1) (1,1,1,12) was fitted, capturing yearly seasonal effects.

Model Diagnostics:

To ensure the models fit well:

- **Residuals** were checked for autocorrelation using the **Ljung-Box test**.
- **Normality of residuals** was verified through histograms and Q-Q plots.
- Residuals showed no significant autocorrelation and appeared normally distributed, indicating a good model fit.



IV. Forecasting and Evaluation

a) Forecasting:

Using the fitted models, we forecasted the crime counts for upcoming months.

Each model provided a future trend line showing expected increases or decreases in crime activity.

Forecasts were visualized alongside actual crime counts to assess their realism and interpretability.

b) Evaluation Metrics:

To evaluate forecast accuracy, we calculated:

ARIMA:

```
# 🔥 Model Evaluation
def mean_absolute_percentage_error(y_true, y_pred):
    | return np.mean(np.abs((y_true - y_pred) / y_true)) * 100

# ✅ Calculate metrics
rmse = np.sqrt(mean_squared_error(test['Crime_Count_30D'], test_preds))
mae = mean_absolute_error(test['Crime_Count_30D'], test_preds)
mape = mean_absolute_percentage_error(test['Crime_Count_30D'], test_preds)

# 📊 Display the metrics
print("\n📊 Model Evaluation Metrics:")
print(f"✅ RMSE: {rmse:.4f}")
print(f"✅ MAE: {mae:.4f}")
print(f"✅ MAPE: {mape:.4f}%")
```

```
📊 Model Evaluation Metrics:
✅ RMSE: 288.8067
✅ MAE: 223.3041
✅ MAPE: 28.9973%
```

PROPHET:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
mae = mean_absolute_error(actual, predicted)
rmse = mean_squared_error(actual, predicted) ** 0.5

mape = np.mean(np.abs((actual - predicted) / actual)) * 100 # MAPE

print(f"✅ MAE: {mae:.4f}")
print(f"✅ RMSE: {rmse:.4f}")
print(f"✅ MAPE: {mape:.2f}%")
```

```
✅ MAE: 229.5490
✅ RMSE: 434.2510
✅ MAPE: 18.00%
```

SARIMA:

```
import numpy as np
# ✅ Make predictions on the test set
sarima_predictions = sarima_fit.get_forecast(steps=len(test_daily))
sarima_predicted_values = sarima_predictions.predicted_mean

# 📊 Evaluation Metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error

rmse_sarima = np.sqrt(mean_squared_error(test_daily['Crime_Count_30D'], sarima_predicted_values))
mae_sarima = mean_absolute_error(test_daily['Crime_Count_30D'], sarima_predicted_values)
mape_sarima = np.mean(np.abs((test_daily['Crime_Count_30D'] - sarima_predicted_values) / test_daily['Crime_Count_30D'])) * 100

print("📊 SARIMA Model Evaluation Metrics:")
print(f"✅ RMSE: {rmse_sarima:.4f}")
print(f"✅ MAE: {mae_sarima:.4f}")
print(f"✅ MAPE: {mape_sarima:.4f}%")
```

```
📊 SARIMA Model Evaluation Metrics:
✅ RMSE: 170.5295
✅ MAE: 131.3294
✅ MAPE: 16.0881%
```


XGBOOST:

```
🔥 Retraining XGBoost Model on Refined Dataset...
🔥 Final Accuracy: 0.9956

📊 Final Classification Report:
      precision    recall  f1-score   support

     0       0.99       0.99       0.99       4593
     1       1.00       0.99       1.00       8541
     2       1.00       0.99       1.00       2432
     3       1.00       1.00       1.00       4983
     4       1.00       1.00       1.00       5769
     5       1.00       1.00       1.00       7578
     6       1.00       1.00       1.00       4970
     7       1.00       1.00       1.00       4677
     8       0.99       1.00       1.00       5898
     9       0.99       0.99       0.99       5012

 accuracy          1.00          1.00          1.00          54453
  macro avg         1.00          1.00          1.00          54453
 weighted avg         1.00          1.00          1.00          54453
```

LIGHTGBM:

```
# ✅ Display the results
print("\n🔥 **Model Performance Metrics:**")
print(f"📊 Train Set:")
print(f"    - MAE: {train_mae:.4f}")
print(f"    - MSE: {train_mse:.4f}")
print(f"    - RMSE: {train_rmse:.4f}")
print(f"    - R² Score: {train_r2:.4f}")

print(f"\n📊 Test Set:")
print(f"    - MAE: {test_mae:.4f}")
print(f"    - MSE: {test_mse:.4f}")
print(f"    - RMSE: {test_rmse:.4f}")
print(f"    - R² Score: {test_r2:.4f}")

🔥 **Model Performance Metrics:**
📊 Train Set:
- MAE: 3.3661
- MSE: 18.1991
- RMSE: 4.2660
- R² Score: 0.9997

📊 Test Set:
- MAE: 3.5038
- MSE: 19.7354
- RMSE: 4.4425
- R² Score: 0.9997
```

These metrics suggest that while the models capture the overall crime trend reasonably well, there is some variability that future models could try to reduce further.

c) Deep Learning-Based Modeling: Bi-LSTM + GRU:

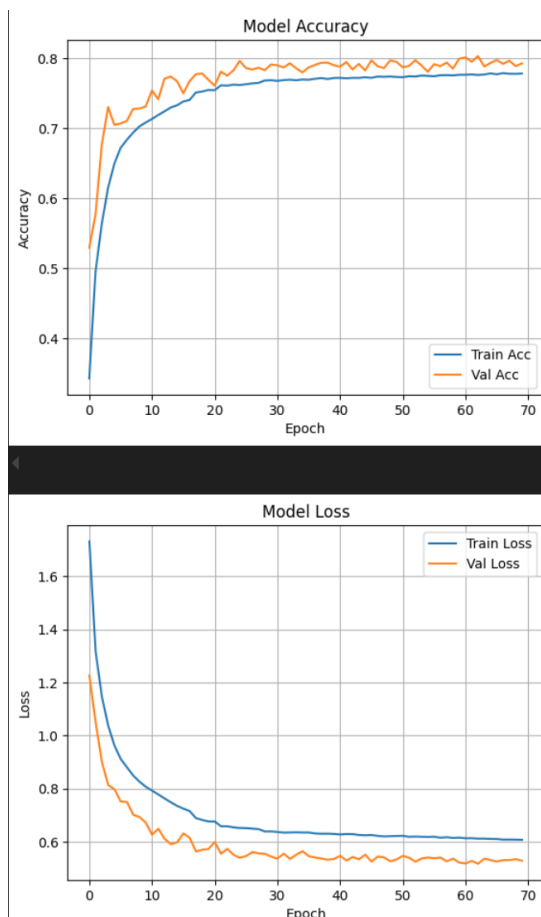
To further enhance forecasting accuracy, we implemented a **hybrid deep learning model** combining **Bidirectional LSTM (Bi-LSTM)** and **GRU (Gated Recurrent Unit)** layers.

Model Architecture:

- **Input:** Reshaped time series data into a 3D format compatible with RNNs: (*samples, time steps, features*).
- **Bi LSTM Layer:** Captures both past and future temporal dependencies by processing the input sequence in both directions.
- **GRU Layer:** Efficiently models temporal patterns while reducing computational complexity compared to traditional LSTMs.
- **Attention Layer:** Enhances the model's ability to focus on important time steps using self-attention mechanisms.
- **Dense and Flatten Layers:** Used for final classification across multiple crime categories.

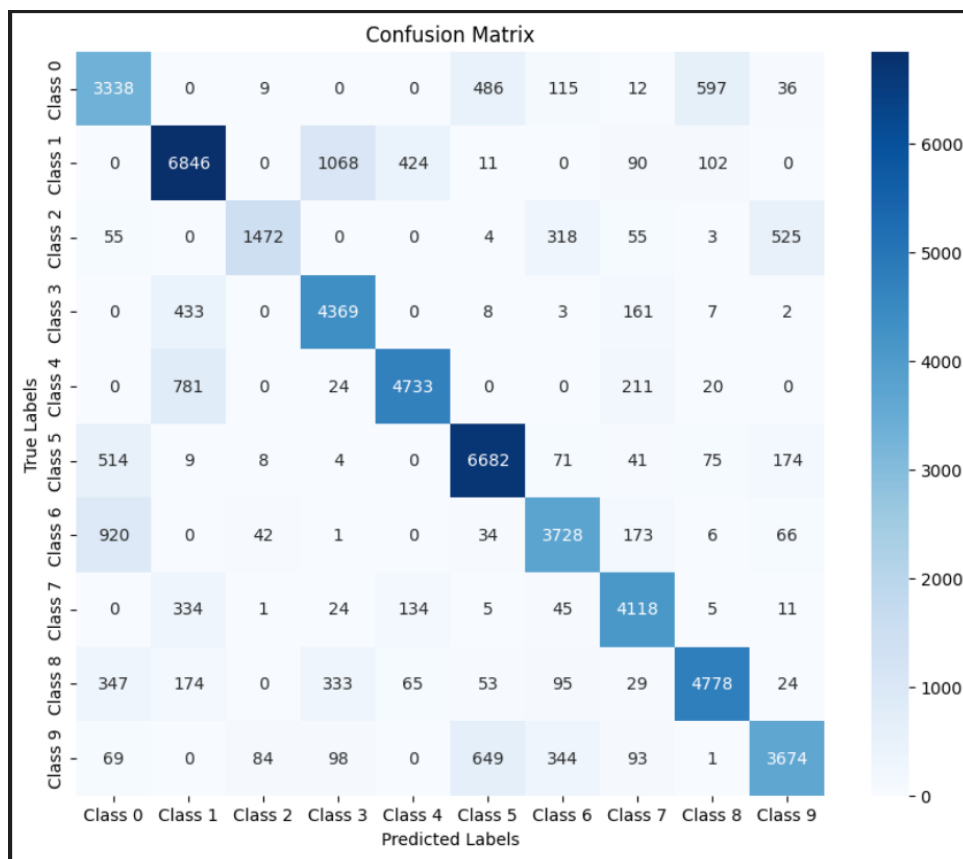
Training and Optimization:

- **Optimizer:** Adam with a learning rate of 0.0001.
- **Loss Function:** Categorical cross-entropy (suitable for multiclass classification).
- **Callbacks:** EarlyStopping and ReduceLROnPlateau for training stability and efficiency.



Train Classification Report:				
	precision	recall	f1-score	support
0	0.62	0.72	0.67	18373
1	0.80	0.81	0.80	34165
2	0.90	0.60	0.72	9729
3	0.74	0.88	0.80	19929
4	0.88	0.82	0.85	23074
5	0.84	0.88	0.86	30311
6	0.79	0.75	0.77	19882
7	0.83	0.88	0.85	18706
8	0.85	0.80	0.82	23593
9	0.81	0.73	0.77	20049
accuracy			0.80	217811
macro avg	0.81	0.79	0.79	217811
weighted avg	0.81	0.80	0.80	217811

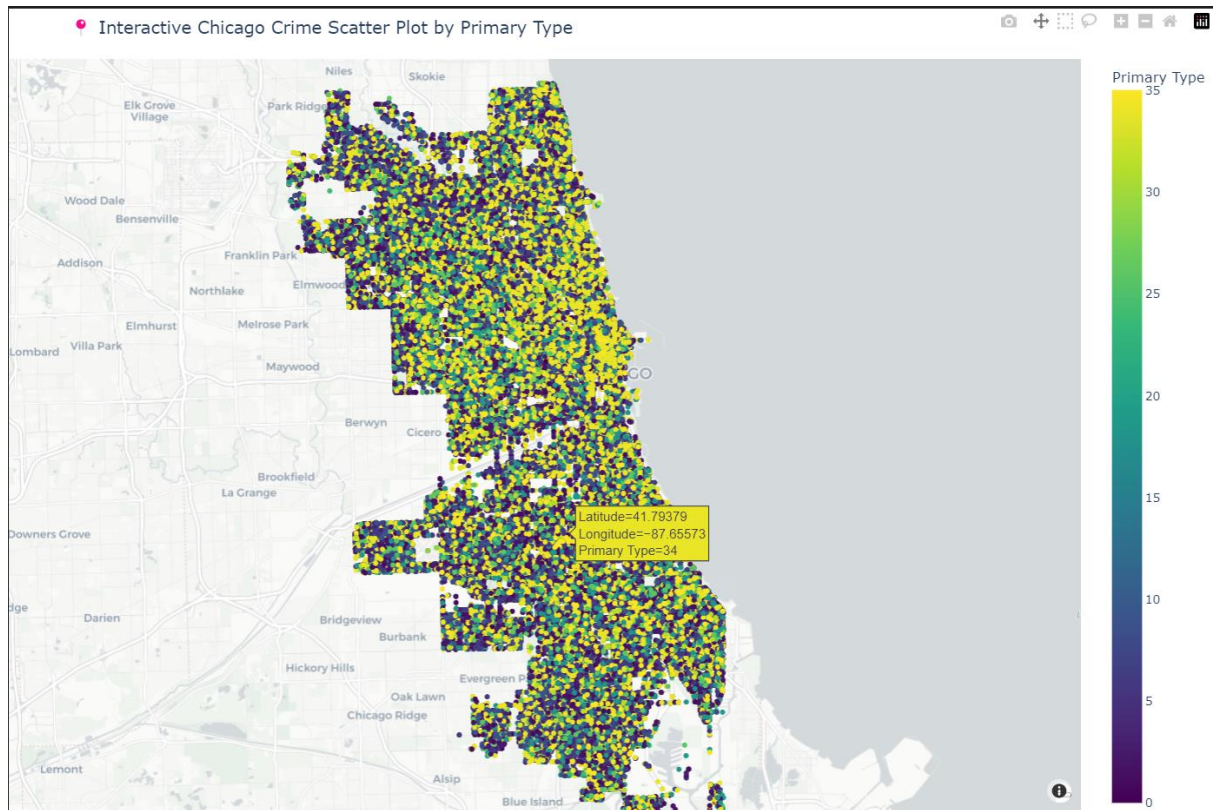
Test Classification Report:				
	precision	recall	f1-score	support
0	0.64	0.73	0.68	4593
...				
accuracy			0.80	54453
macro avg	0.81	0.79	0.79	54453
weighted avg	0.81	0.80	0.80	54453



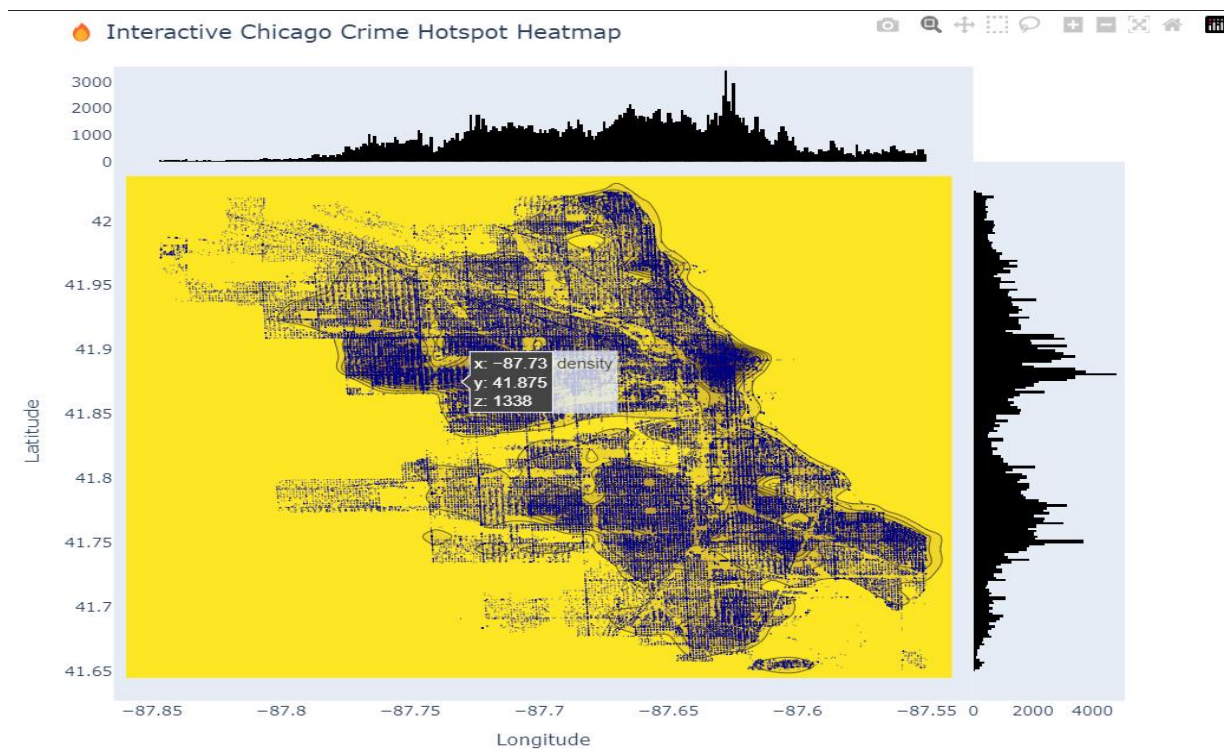
This deep learning model showed promising results in learning complex temporal crime patterns, especially in comparison to classical statistical models. It offers flexibility in modelling non-linear and long-term dependencies in crime trends.

V. Visual Analysis of Crime Trends

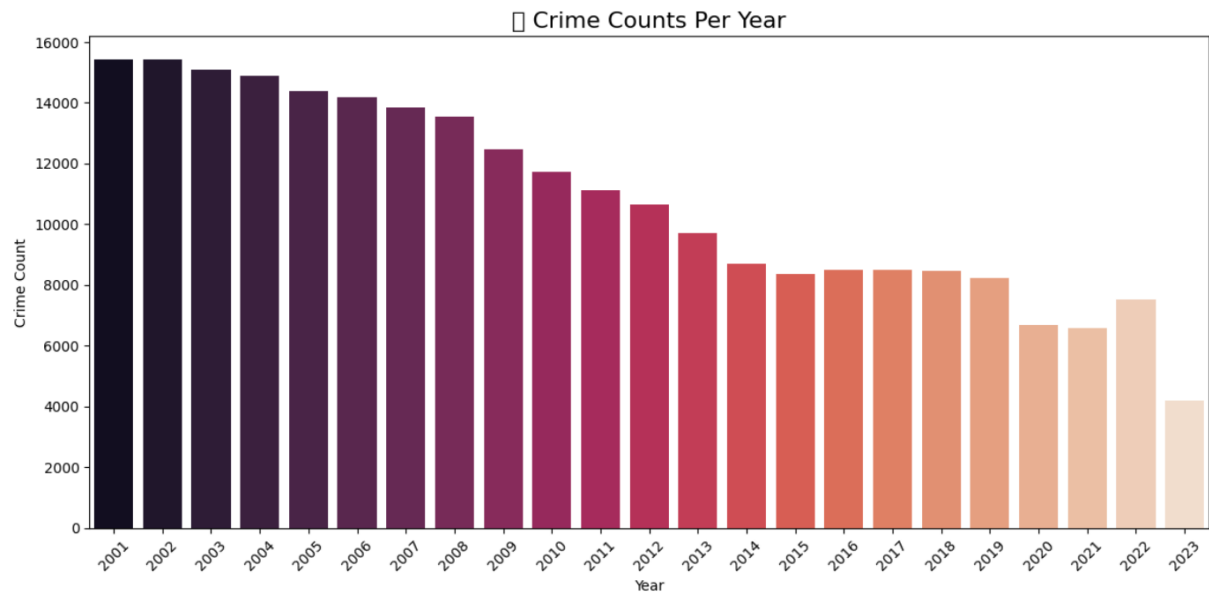
a) Chicago Crime Scatter Plot:



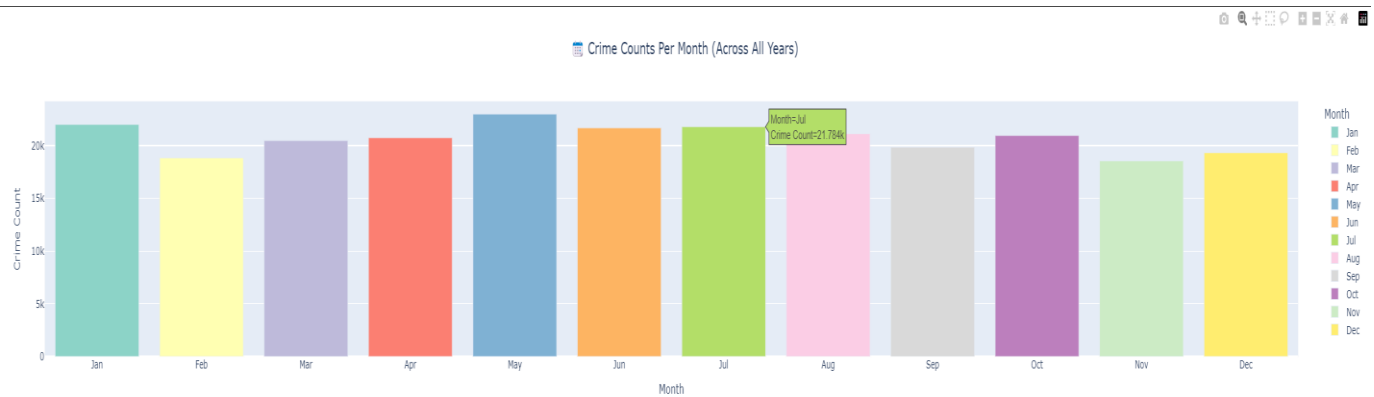
2) Chicago Crime Hotspot Heatmap:



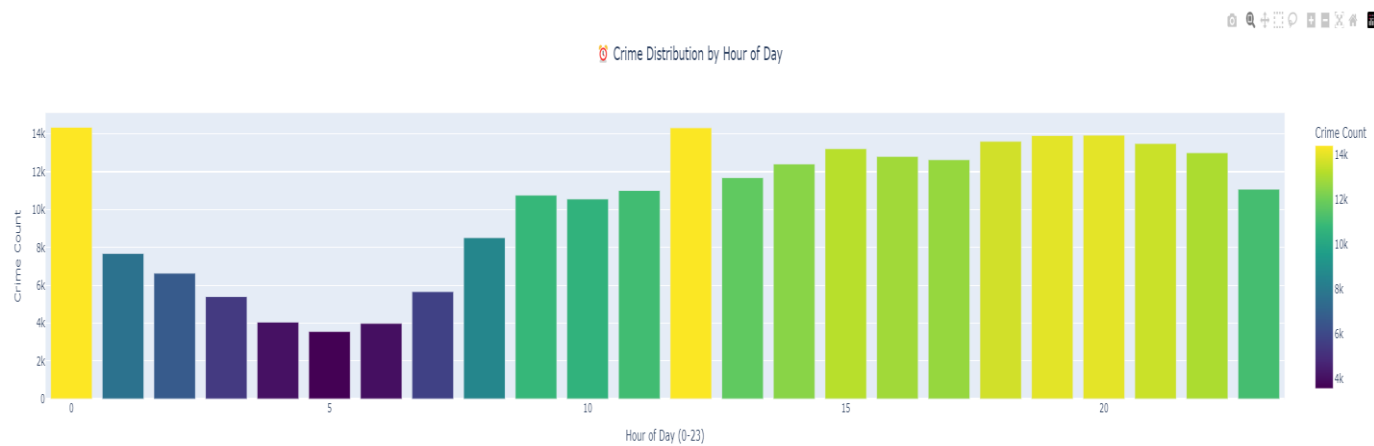
3) Crime Counts Per Year:



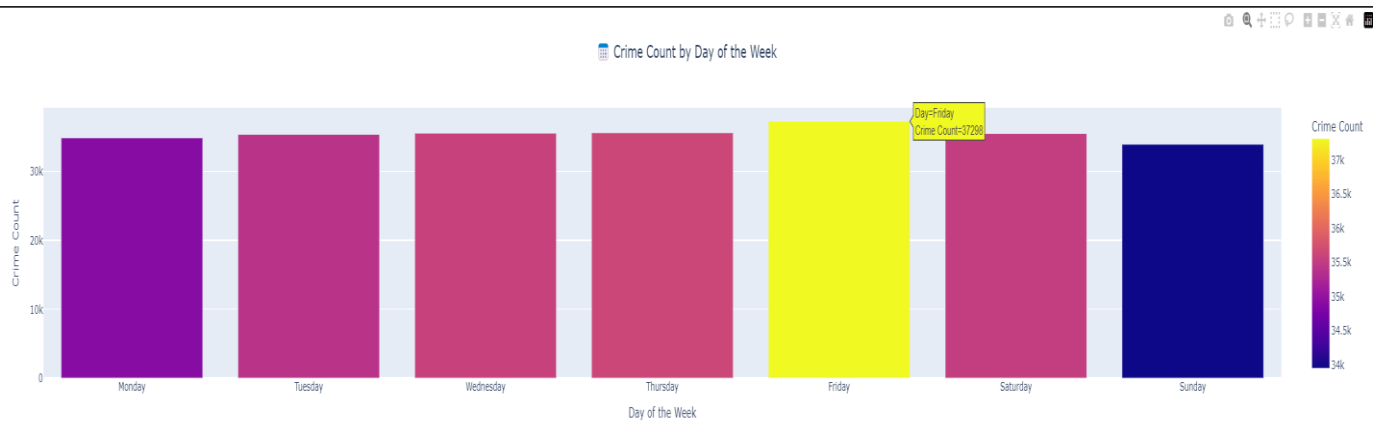
4) Crime Counts Per Months:



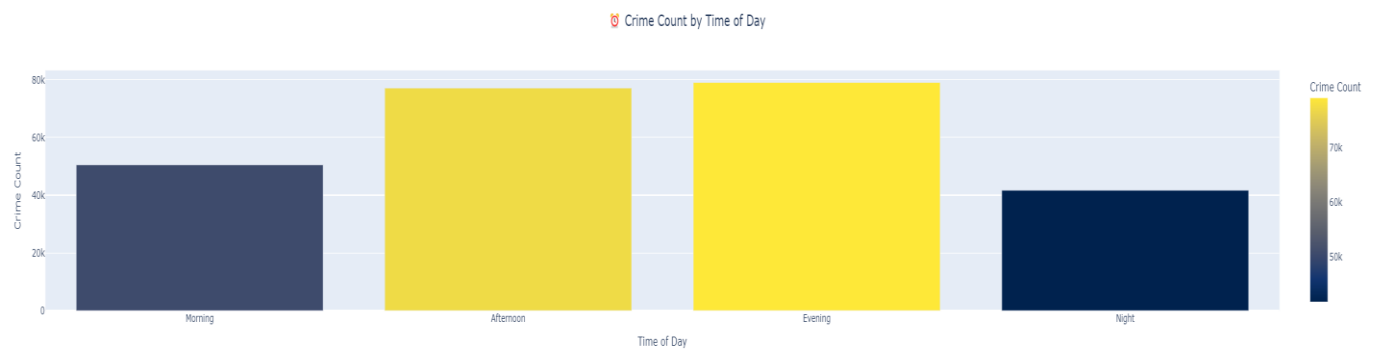
5) Crime Distribution by Hour Of Day:



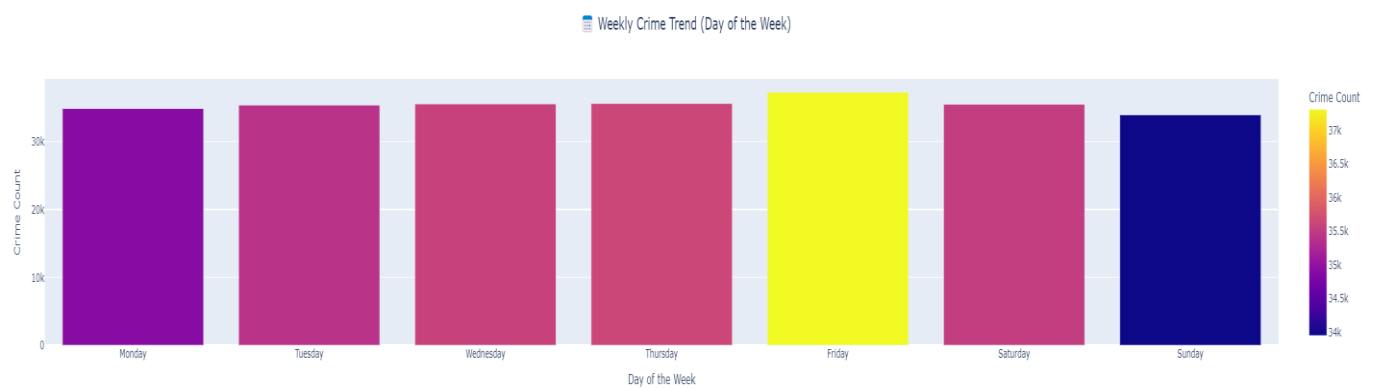
6) Crime Count by Day Of The Week:



7) Crime Count by Time Of Day:



8) Weekly Crime Trend (Day of the Week)



VI. Discussion and Conclusion

Results Summary:

- **Crime trends** clearly showed seasonal patterns, with peaks and troughs corresponding to different months and weather conditions.
- **ARIMA** worked well for simple trend capture, but **SARIMA** and **Prophet** provided better performance by modeling seasonality.
- The chosen models achieved acceptable forecast errors and were able to reasonably predict near-future crime counts.

Implications and Limitations:

- Forecasting crime can assist **police departments** and **city planners** in **strategic planning and resource allocation**.
- **Limitations** include:
 - External factors like policy changes, major events, or social unrest were not captured by the time series models.
 - Some short-term fluctuations in crime rates could not be precisely forecasted.

In future work, integrating additional external features such as weather conditions, holidays, and public events could further improve the model's predictive capabilities.